Ukrainian Catholic University

Bachelor Thesis

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

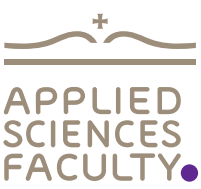
Author:
Radomyr Husiev

Supervisors:
Oleg Farenyuk
Oleksandr Kosovan

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences



Lviv 2026

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

by Radomyr Husiev

Abstract

TODO

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Ground vehicles – very important field; especially for tedious, far-away, or dangerous tasks.

It is often infeasible to manually control such vehicles due to their sheer number and task volume. Physical barriers like distance or signal interference mean manual control can fail. Therefore, autonomous systems are a necessity for the robot to complete its task and navigate to desired locations independently.

Outdoors, there are challenges of an unpredictable environment: unexpected/moving obstacles, surrounding agents (like other vehicles or pedestrians) don't cooperate, uneven terrain, etc.

Today, there are ready-made solutions for autonomous navigation (like default Nav2 in ROS). However, modern outdoor systems often rely on heavy sensors (3D LiDAR, RGB-D cameras) and algorithms to process this data (Visual SLAM, MPPI). These require high computational resources, lots of RAM, and sometimes GPUs. This is expensive, it drains battery and not always possible on simple low-cost robots.

Therefore, there is a motivation to see what can be accomplished using a minimal, lightweight sensor suite. By avoiding heavy computations, we can create a more accessible and energy-efficient system.

Instead of calculating a map from scratch using SLAM, we can use existing, semantic data like OpenStreetMap (OSM) for global routing. Instead of 3D point clouds or image processing, we can use a simple 2D LiDAR to handle immediate, dynamic obstacles (like pedestrians or trees). Instead of visual odometry, we can rely on standard GPS fused with an IMU and wheel odometry.

Together, these lightweight components theoretically provide all the necessary data for a robot to navigate (albeit in lower fidelity and density, higher sparcity). This thesis aims to prove that such a minimal setup is actually viable in the real world.

The setup includes a ground differentially driven robot outdoors in a park environment, specifically on pedestrian paths in a known (topologically, but not dynamically) environment. It should navigate autonomously from a start point to a target point without human intervention.

1.2 Objectives

- Path planning – find a global route (using graph search algorithms like A*) based on environment map (OpenStreetMap XML data).
- Obstacle detection – process 2D LiDAR data to find obstacles.
- Local navigation – combine spatial localization (GPS and IMU+Wheel Odometry – sensor fusion via Extended Kalman Filter) and local path planning (using

an optimization algorithm Timed Elastic Band) to avoid obstacles and follow the global path.

Conducted real-world experiments in public environment (Stryyskyi Park). Also in simulation – more times here. Evaluated the system's success rate, safety, and travel time. Provided a baseline comparison against the standard Nav2 implementation.

1.3 Structure Of The Thesis

Chapter 2

Related Work

2.1 Path and Trajectory Planning

The problem of autonomous navigation classically divided into:

- Global path planning
- Local trajectory planning/obstacle avoidance

Global – graph-based approaches like Dijkstra's or A*, because reliable and optimal on static maps.

But when moving in dynamic environments, real-time local planning is required. Early approaches, such as the Artificial Potential Fields (APF) method, modeled the environment as a physical field: the goal attracts, the obstacles repel. But it suffered from local minimums and worked worse with tight gaps. Later, the Vector Field Histogram (VFH) was introduced: simpler and with better performance, though it struggled with complex paths and kinematic constraints of vehicles.

There is a widely used Dynamic Window Approach (DWA), which takes a set of feasible new velocities for the next few seconds, checks each one for obstacles, goal etc. → chooses the best. The resulting trajectory is addresses the kinematics (smooth geometry of the path), but it struggles to produce paths for complex environments. Therefore, it is included in ROS NAV2 stack, but as a simpler and less powerful planner.

Then there is Elastic Band planner, which tries to predict the whole path as a band between two points, which stretches to avoid obstacles. But it does not produce a trajectory, it only provides with a geometric path (instead of telling the controller what speeds to drive at what points in time, it only provides a path – a sequence of points the robot should follow).

Timed Elastic Band (TEB) is a continuation, which shifts toward the Model Predictive Control (MPC) paradigm. MPC-based algorithms optimize a continuous trajectory, while enforcing kinodynamic constraints – the geometrical and physical (acceleration, forces etc.) constraints. TEB optimizes the trajectory for time and distance to obstacles. More recently, sampling-based MPC algorithms like Model Predictive Path Integral Control (MPPI) have emerged for aggressive, high-speed driving. NAV2 once used TEB for its advanced trajectory planner, but recently moved to MPPI.

While MPC methods generate superior trajectories compared to reactive methods like DWA, they require higher computational overhead. MPPI in particular requires parallel sampling that often necessitates a GPU, making it unsuitable for constrained systems. Therefore, a CPU-optimized MPC approach like TEB provides a middle ground between performance and efficiency.

2.2 Localization

You need to know the location of the robot to know where to drive. Currently one of the most used methods is Simultaneous Localization and Mapping (SLAM), particularly Visual-Inertial SLAM (VI-SLAM) and 3D LiDAR-based SLAM. However, these methods require lots of computational power. For example [Schmidt] demonstrated that VI-SLAM algorithms can use 4-6 GB of RAM for the mapping node and their accuracy degrades significantly if the frame rate is lowered to save processing power. Furthermore, outdoor environments—specifically parks with heavy vegetation—cause SLAM algorithms to drift or lose tracking entirely due to dynamic foliage and lack of distinct structural features.

Because of these limitations, using Extended Kalman Filter (EKF) to fuse GPS, Inertial Measurement Units (IMU) and wheel odometry can be used as a stable alternative. [Moore] showed that while wheel odometry drifts immensely over time, fusing it with IMU provides an improvement. Adding GPS, even if jumpy and unpredictable under trees, further reduces the long-term drift, providing a reliable localization without the overhead of SLAM.

2.3 Semantic Mapping

SLAM is not only for localization, but also for mapping. To further avoid the costs of SLAM, researches explored using existing semantic maps for global planning. [Chen] note a growing trend of reducing SLAM dependency by navigating via pre-existing semantic data.

OpenStreetMap (OSM) is fit for this use case. [Hentschel] showed that autonomous navigation can be performed using OSM geodata. There's also a benefit of OSM over SLAM, because it provides the semantic context of the environment, distinguishing pedestrian path and grass, describing the surface materials etc. Furthermore, [Stahr] utilize OSM attributes such as path width to create boundaries for local planners, which limits the search space for trajectory planning algorithms like TEB.

2.4 Obstacle Abstraction

Even with a lightweight mapping approach, processing raw sensor data for immediate obstacle avoidance remains resource-intensive. Compared to 2D LiDAR, 3D one has much more points (a vertical dimension). With 3D LiDAR there also emerges a problem of ground points, which need to be filtered out in order not to consider the roal itself as an obstacle. Due to these limitations 3D LiDAR becomes computationally expensive.

Using 2D LiDAR solves the vertical complexity but still produces hundreds of data points per scan. To optimize this, researchers use algorithms to extract geometric primitives out of point clouds. [Catapang] and [Nguyen] walked through the scan and grouped all points at the distance lower than the vehicle diameter into clusters. Then to extract the clusters into geometric primitives [Sezer] used circle approximation. [Nguyen] compared several algorithms and found that the "Split and Merge" approach is highly efficient and accurate for converting raw LiDAR points into geometric lines. Sometimes the approaches are combined like in [Catapang], extracting a different primitive (circle, line or rectangle) depending on the cluster shape.

Some approaches treat other agents differently from regular obstacles. However, implementations such as [Du] require Reinforcement Learning and 3D tracking, which conflicts with computational efficiency.

2.5 Conclusion

Chapter 3

Theoretical Background

3.1 Differential Drive Robots

In this thesis the robot uses the four-wheeled skid-steer differential drive configuration. While the machine has four points of contact, the steering is achieved through a velocity differential between the left and right wheel pairs.

This system can be treated the same way as a two-wheeled differential drive system. It means that robot's movement has non-holonomic constraints: it can rotate in place but cannot move laterally – instead the robot's movement is controlled by linear (v) and angular (ω) velocities. Planners must respect these constraints. A generated trajectory lacking feasible (v, ω) pairs cannot be executed by the hardware.

Unlike two-wheeled systems, the four-wheeled configuration introduces lateral slip (skidding) during rotation, which increases drift in wheel-based odometry. Thus, IMU and GPS become more important to correct for the drift.

3.2 Spatial Representation

3.2.1 Coordinate Systems

GPS sensors output data in geodetic coordinates (latitude, longitude, altitude). Trajectory planners operate in Cartesian coordinates. Geodetic coordinates must be projected onto a local tangent plane in order for local planner to calculate distances in meters.

3.2.2 Graph-Based Map Representation

For global planner to generate paths, it needs the map to be abstracted into a directed graph where vertices represent discrete spatial coordinates (intersections) and edges represent traversable paths (roads). The nodes in this network exist only at intersections. Complex paths made of multiple segments are simplified into single edges connecting these intersections. The weight of each edge is the sum of all its original segment lengths. This preserves distance accuracy while reducing the number of nodes the planning algorithm must evaluate.

The A* algorithm navigates this graph. It calculates the shortest path by minimizing the function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost from the start node to node n and $h(n)$ is the estimated heuristic cost. For A* to guarantee the mathematically shortest path, the heuristic must be admissible, meaning it must never overestimate the actual cost to the goal. Because a straight line is always the shortest possible distance between two points, the Euclidean heuristic is admissible.

With the heuristic cost in place, the algorithm first processes paths closer to the goal, thus the frontier of the search stretches in an elliptical shaped, pointing at

the target. This naturally reduces the number of nodes processed, while preserving accuracy.

3.3 Trajectory Optimization

The Timed Elastic Band (TEB) approach formulates local navigation as a multi-objective optimization problem.

The trajectory is a sequence of intermediate poses between them. Each pose consists of x, y coordinates (in meters), θ (heading angle, in radians) and Δt (time interval, in seconds). The trajectory is initialized as the global path, with Δt and θ linearly interpolated.

The trajectory is then deformed based on constraints and obstacles. The problem is framed as a non-linear least squares problem to be optimized. The robot's poses act as the parameters. The physical and spatial constraints act as cost functions (or residuals). The residuals might include obstacle distance cost, kinematic cost, time cost. The optimization engine minimizes a weighted sum of these cost functions.

Minimizing these costs requires calculating Jacobians – matrices of partial derivatives indicating how changes in the trajectory parameters affect the total cost. These can be calculated analytically (either manually or automatically) or numerically (using finite differences at runtime).

A solver like Ceres (or g2o, if framing the problem as a Hyper-Graph optimization) uses Jacobians to find gradient of the cost functions. Because trajectory constraints usually affect adjacent poses, the resulting mathematical system is highly sparse. Ceres applies sparse factorization methods (such as Cholesky decomposition) to efficiently solve the equations, iteratively updating the parameters until it converges on a smooth, physically feasible trajectory.

3.4 Sensor Fusion

Sensors contain noise and inherent inaccuracies. Wheel odometry accumulates drift from mechanical slippage, while IMU exhibits integration drift. Small offsets in IMU sensor bias and noise result in quadratic position error growth and orientation errors caused by gravity leakage. Global Positioning System (GPS) sensors provide absolute global coordinates but suffer from high-frequency noise and interference under tree cover or near structures.

The Kalman Filter fuses sensor streams into a single reliable state estimate. It maintains a state vector and a covariance matrix, which represents the system's current uncertainty for each sensor. It uses IMU and wheel odometry to continuously predict the robot's next state, but with time uncertainty grows. GPS is used to correct the predicted state upon measurement and reduce uncertainty.

The standard Kalman Filter assumes linear system dynamics. However, mobile robots involve non-linear movements, such as turning and circular trajectories, which require trigonometric functions. The Extended Kalman Filter addresses this by linearizing motion and models at the current state estimate (achieved through Taylor series expansion).

This fusion also ensures that even if a sensor like GPS becomes temporarily unavailable, the robot can continue to navigate using its internal motion model until the absolute signal is restored.

3.5 Geometric Feature Extraction

2D LiDAR outputs data in polar coordinates (distance and angle). To be used by the optimization engine, this point cloud must be segmented and abstracted into geometric primitives (lines and circles) in Cartesian space.

3.5.1 Clustering

Point clouds are first grouped into discrete clusters using distance-based segmentation. Points are assigned to a single object if the Euclidean distance between consecutive measurements P_i and P_{i+1} is below a threshold. Isolated points or clusters with insufficient density are rejected as noise.

3.5.2 Line Fitting

Line extraction uses the Split-and-Merge algorithm. First, a chord is drawn between the cluster's endpoints. The algorithm calculates the perpendicular distance from every point to this chord. If the maximum distance exceeds an error threshold, the cluster is split at that point into two sub-scans, and the process recurses. The result is a set of linear segments that approximate complex shapes like walls or curbs.

3.5.3 Circle Fitting

For clusters identified as obstacles more similar to circles (such as trees, poles, or people), geometric parameters (center and radius) are extracted using another algorithm. It selects three points: the two extremes (P_1, P_3) and a central point (P_2) that best represents the arc's curvature. Perpendicular bisectors are constructed for the secant lines P_1P_2 and P_2P_3. The intersection of these bisectors defines the circle's center. This method provides an analytical solution for the center.

Chapter 4

Proposed Solution

4.1 Problem Formulation

4.2 Contributions

Developed a ROS2 navigation pipeline for outdoor pedestrian paths.

Parse OSM and use road attributes like surface to filter.

Virtual corridor.

UI; split architecture UI and controller.

Implemented a filtering mechanism that translates raw point clouds from 2D LiDAR into geometric primitives (circles, lines) for a reduced computational cost of TEB.

4.3 Local Planner Waypoints

The result of the global search is a sequence of intersections the robot should follow. To get the path polyline (a sequence of waypoints $W = \{w_1, w_2, \ldots, w_n\}$ connected by straight segments), each edge is expanded using OSM data. The waypoints are then used as a reference for the local planner. Additionally, a navigable corridor may be generated

Chapter 5

Experiments and Results

Chapter 6

Conclusions

Appendix A

Appendix